

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第三章 句法分析

3.1 句法分析概述

3.2 成分句法分析

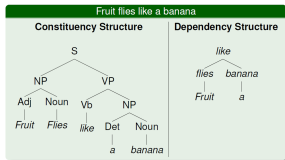
3.3 依存句法分析

3.4 句法分析语料库

第三章句法分析

引言

- ▶ 句法分析 (Syntactic Parsing) 是自然语言处理 (NLP) 中的一个关键领域，旨在通过自动化的方式解析句子的结构，识别句子中的成分（如主语、谓语、宾语等）以及它们之间的语法关系。句法分析的主要目标是生成句子的句法树 (syntactic tree)，这种树状结构能够清晰地展示句子的语法层次和成分之间的关系。



以句子” Fruit flies like a banana.” 为例，我们可以通过句法分析来解析其结构和语法关系。这个句子有两种可能的解释，因此句法分析的结果也会有所不同。第一种解释：果蝇喜欢香蕉；第二种解释：水果像香蕉一样飞。这个句子之所以有两种解释，是因为” fruit flies” 可以被理解为一个复合名词（果蝇），也可以被分开理解为” fruit”（水果）和” flies”（飞）。这种歧义性在自然语言中非常常见，句法分析需要结合上下文或语义信息来消除歧义。

引言

► 1. 语言的无限性与句法的规则性

- 语言的无限性：人类语言具有递归性（recursion），即通过有限的规则和词汇，可以生成无限数量的句子。例如，通过添加形容词、副词、从句等，句子可以无限扩展：
- 简单句：The cat sleeps.
- 扩展句：The small cat sleeps peacefully on the soft mat.
- 更复杂句：The small cat, which I adopted last year, sleeps peacefully on the soft mat that I bought from the store.
- 句法的规则性：尽管句子可以无限扩展，但它们并非随意组合，而是遵循特定的语法规则。这些规则确保了句子的合法性和可理解性。

► 2. 句法的核心作用

- 句法规则的作用：
- 词序规则：不同语言有不同的词序规则。例如，英语是主谓宾（SVO）结构，而日语是主宾谓（SOV）结构。
- 短语结构规则：句法规则定义了如何将词组合成短语（如名词短语、动词短语等），以及如何将短语组合成句子。
- 层级结构：句子并非线性排列，而是具有层级结构。例如，句子“The cat on the mat sleeps”中，“on the mat”是修饰“the cat”的介词短语，而不是直接与“sleeps”相关。
- 句子的合法性：句法规则可以判断一个句子是否符合语法。例如，“The cat sleeps”是合法的，而“Sleeps cat the”则不符合英语语法。

▶ 3. 句法与语义的关系

- ▶ 句法影响语义：句法规则不仅决定了句子的结构，还影响了句子的意义。例如：

“The man saw the woman with the telescope.” 这句话有两种可能的解释：

- ▶ 男人用望远镜看到了女人。
- ▶ 男人看到了带着望远镜的女人。
- ▶ 这种歧义性源于句法结构的不同解析方式。
- ▶ 语义约束句法：语义信息也可以帮助消除句法歧义。例如，在 “Fruit flies like a banana” 中，语义知识可以帮助确定 “fruit flies” 是指果蝇，而不是水果在飞。

3.1 句法分析概述

基本概念

- ▶ **语法 (Grammar)** 就是指自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律。根据语法就可以判断不同成分组成句子的方式以及决定句子是否成立。广义的语法包括句法 (Syntax)、形态学 (Morphology)、语义学 (Semantics) 和语音学 (Phonology) 等多个层面。
- ▶ **狭义的语法学研究基本等同于句法学，研究句子、短语和词的结构规则。**句法是语法的核心部分，研究如何将词组合成短语，再将短语组合成句子，并确定句子的合法性。
- ▶ 句法是现代语言学研究中的重要课题，有大量的**句法理论 (Syntactic Theory)** 相关研究。自 19 世纪 50 年代以来，语言学家们构建了大量明确且形式化的语法理论，为自然语言句法分析提供了理论支持。在构建语法理论时，一个重要的问题是该理论是基于成分关系还是基于依存关系。

成分语法理论概述

- ▶ 诺姆·乔姆斯基 (Noam Chomsky) 于 1957 年发表的《Syntactic Structures》中提出了生成语法 (Generative Grammar), 奠定了成分语法的基础。
- ▶ 核心思想：句子是由一系列嵌套的短语结构组成的，这些短语结构可以递归地生成无限数量的句子。
- ▶ 成分 (Constituent) 又称短语结构，是指一个句子内部的结构成分。成分可以独立存在，或者可以用代词替代，又或者可以在句子中的不同位置移动。
- ▶ 短语结构规则：成分语法使用形式化的规则描述句子的结构。例如：
S $\rightarrow$ NP VP (句子由名词短语和动词短语组成)
NP $\rightarrow$ Det N (名词短语由限定词和名词组成)
VP $\rightarrow$ V NP (动词短语由动词和名词短语组成)
“他正在写一本小说。”
“一本小说”是一个成分

成分语法理论概述

- ▶ **句法范畴 (Syntactic Category)** 是指根据成分在句子中的语法功能和分布特性，将成分划分为不同的类别。同一句法范畴的成分可以在句子中相互替代而不影响句子的语法正确性。

“一本小说”和“一所大学”都属于名词短语 (NP)，因为它们可以在句子中相互替代：

他正在读一本小说。→ 他正在读一所大学。

虽然语义上不合理，但句法上是合法的。

- ▶ 句法范畴不仅仅包含名词短语 (NP)、动词短语 (VP)、介词短语 (PP) 等短语范畴，也包含名词 (N)、动词 (V)、形容词 (Adj) 等**词汇范畴**。除此之外还包含**功能范畴**（包括冠词、助动词等）。

成分语法理论概述

- ▶ 句法范畴之间不是完全对等的，而是具有**层级关系**。

例如：一个句子可以由一个名词短语和一个动词短语组成，一个名词短语可以由一个限定词和一个名词组成，一个动词短语又可以由一个动词和一个名词短语组成。

- ▶ **短语结构规则 (Phrase Structure Rules)** 又称改写规则或重写规则，对句法范畴间的关系进行形式化描述。

短语结构规则通常表示为 $X \rightarrow Y Z W \dots$ ，其中：

X 是一个句法范畴（如 NP、VP 等）。

→ 表示“改写为”或“由... 组成”。

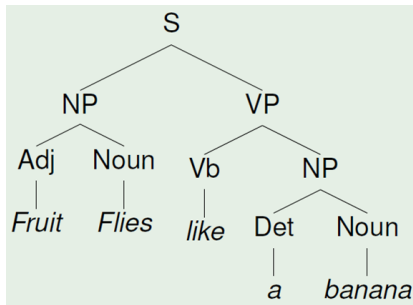
Y Z W ... 是构成 X 的成分，可以是词汇范畴（如 N、V）或其他短语范畴（如 NP、VP）。

成分语法理论概述

例如：

- ▶ $S \rightarrow NP VP$ (句子由名词短语和动词短语组成)
- ▶ $NP \rightarrow Adj N$ (名词短语由形容词和名词组成)
- ▶ $VP \rightarrow V NP$ (动词短语由动词和名词短语组成)
- ▶ $NP \rightarrow Det N$ (名词短语由限定词和名词组成)

这种层级性使得句法分析可以递归地进行，即一个短语可以包含另一个同类型的短语。这种递归性使得语言能够生成无限数量的句子。



成分语法理论概述

- ▶ **成分语法**就是由句法范畴以及短语结构规则定义的语法。由于短语结构规则具有递归性，可以使短语和句子无限循环组合。这也说明了语言的创造性和无限性。
- ▶ 成分语法的优势：**形式化描述**：成分语法提供了一种形式化的方法描述句子的结构，便于计算机处理。**层级结构**：成分语法通过句法树清晰地展示句子的层级结构，便于理解和分析。成分语法在 NLP 中具有广泛的应用，如句法分析、语言生成等。
- ▶ 由于成分语法**局限于表层结构分析**，不能彻底解决句法和语义问题，因此存在**非连续成分**(即一个成分被其他成分分隔开。成分语法难以处理这种情况。“What did you see?” “What” 是动词 “see” 的宾语，但在句子中被移到句首，形成了非连续成分。)、**结构歧义**(同一个句子可能有多种合法的句法解析，成分语法无法完全消除这种歧义。)、**语义问题** (成分语法主要关注句子的表层结构，无法直接处理语义问题。例如，成分语法无法解释为什么某些句子在句法上合法但在语义上不合法。) 等问题。

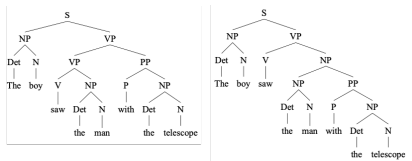


图: 句子 The boy saw the man with telescope 的两种成分句法树

依存语法理论概述

- ▶ 依存语法 (Dependency Grammar) 的核心思想及其在句法学中的重要地位。依存语法由法国语言学家泰尼埃尔 (Lucien Tesnière) 在 1959 年发表的《Éléments de syntaxe structurale》中提出，是一种以中心词和依存关系为核心的句法分析方法。这个理论奠定了依存语法研究的基础。
- ▶ 依存语法的核心思想
- ▶ 中心词 (Head)：每个句子或短语都有一个中心词，它是句法结构的核心，其他成分都直接或间接依赖于它。
- ▶ 依存关系 (Dependency Relation)：句子中的成分通过依存关系连接，形成一种树状结构。依存关系是二元的、非对称的，并且具有方向性：
一个成分是中心语 (Governor)，另一个成分是依存成分 (Dependent)。
依存关系从中心语指向依存成分。

依存语法理论概述

中心成分是句法结构的核心，其他成分都直接或间接依赖于它。

中心成分可以分为以下几种：

- ▶ **Governor (支配者)** 通常指句子中一个支配其他词语的中心词，它可以是动词、名词、形容词等，而其他被支配的词语称为 Dependent。

在句子“男孩吃苹果。”中，动词“吃”是 Governor，支配主语“男孩”和宾语“苹果”。

在短语“漂亮的女孩”中，名词“女孩”是 Governor，支配形容词“漂亮”。

- ▶ **Regent (支配者)** 通常指在句子中代表某个成分的中心词，例如代词或名词，在句子中扮演着某种角色。

在句子“他正在读书。”中，代词“他”是 Regent，代表主语。

在句子“这本书很有趣。”中，名词“书”是 Regent，代表主语。

- ▶ **Head (中心词)** 通常指短语结构中的中心词，例如名词短语中的名词，动词短语中的动词。

在名词短语“一本有趣的书”中，名词“书”是 Head。

在动词短语“正在读书”中，动词“读”是 Head。

依存语法理论概述

依存成分是依赖于中心成分的词或短语，可以分为以下几种：

- ▶ **Modifier (修饰词)** 是用来修饰中心成分的词或短语，描述中心成分的性质、状态等。

在“漂亮的女孩”中，“漂亮”是 Modifier，修饰“女孩”。

在“非常快”中，“非常”是 Modifier，修饰“快”。

- ▶ **Subordinate (从属者)** 指的是从句，即在主句中出现的完整的句子，它不能独立存在，必须依赖于主句。

在句子“我知道他在读书。”中，从句“他在读书”是 Subordinate，依赖于主句“我知道”。

在句子“如果你来，我会很高兴。”中，从句“如果你来”是 Subordinate，依赖于主句“我会很高兴”。

- ▶ **Dependency (依存关系)** 指的是语法中的依存关系，它描述了一个单词或短语如何依赖于另一个单词或短语来形成句子。依存关系包括修饰关系和从属关系。

在句子“男孩吃苹果。”中，名词“男孩”和“苹果”都依赖于动词“吃”。

在短语“在桌子上”中，名词“桌子”依赖于介词“在”。

依存语法理论概述

有向图表示：用有向弧表示依存关系，支配者（中心语）位于弧的发出端，被支配者（依存成分）位于弧的箭头端。

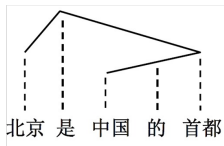
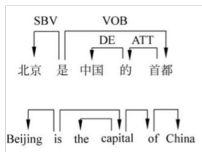
► 特点：直观地展示了词与词之间的依存关系。适用于表示复杂的依存关系（如非投射性结构）。

► 句子：“男孩吃苹果。”

吃 → 男孩

吃 → 苹果

“吃”是支配者，“男孩”和“苹果”是被支配者。



依存语法理论概述

依存树 (Dependency Tree)：是一种树状结构，其中：父节点是支配者。子节点是被支配者。

- ▶ 特点：树状结构清晰地展示了句子的层级关系。每个节点只有一个父节点（除了根节点）。

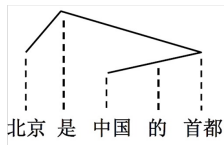
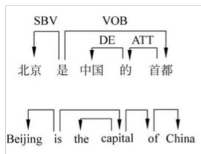
- ▶ 句子：“男孩吃苹果。”

吃（根节点）

├—— 男孩（子节点）

└—— 苹果（子节点）

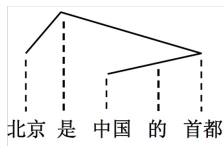
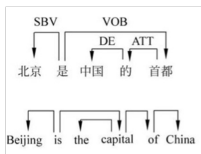
“吃”是根节点，“男孩”和“苹果”是子节点。



依存语法理论概述

依存投射树 (Projective Dependency Tree)：是一种特殊的依存树，其中：实线表示依存关系。虚线表示投射线，用于确保句子中的词在投影后不交叉。位置低的成分依存于位置高的成分。

- ▶ 特点：投射树要求句子中的词在投影后不交叉，因此只能表示投射性结构。投射树对句子的结构表达能力更强，能够更清晰地展示词与词之间的层级关系。
- ▶ 投射线确保句子中的词在投影后不交叉。



依存语法理论概述

两个单词之间是否存在依存关系？单词之间谁处于支配地位？谁处于从属地位？建立这些词与词之间关系的依据是什么？

- ▶ 依存关系的判断: 判断两个单词之间是否存在依存关系，以及谁处于支配地位、谁处于从属地位，主要依据以下原则：
句法规则：根据语言的句法规则，确定词语之间的合法组合方式。
语义角色：根据词语的语义角色（如施事、受事、工具等），确定它们之间的关系。

配价 (Valency) 理论：根据词语的配价能力，确定它能支配多少个行动元（名词词组）以及这些行动元的语义角色。

- ▶ 配价理论的核心概念

价 (Valency)：价是词语的一个属性，表示它与其他词语结合的能力。价的数目决定了词语能支配多少个行动元。例如，动词“吃”的价数为 2，因为它需要支配一个主语（施事）和一个宾语（受事）。

行动元 (Actant)：行动元是动词所支配的名词词组，通常包括主语、宾语等。例如，“男孩吃苹果。”，“男孩”和“苹果”都是动词“吃”的行动元。

依存语法理论概述

- ▶ **配价模式 (Valency pattern)** 则是描述了某一个具有特定意义的词的出现语境，以及当一个词出现在一个特定的模式下时，还有哪些词语会出现在这个模式下及其语义角色。

- ▶ 例子：

动词“给”的配价模式：

主语（施事）：“我”

间接宾语（接受者）：“你”

直接宾语（受事）：“书”

例句：“我给你书。”

动词“放”的配价模式：

主语（施事）：“他”

直接宾语（受事）：“书”

介词短语（地点）：“在桌子上”

例句：“他把书放在桌子上。”

依存语法理论概述

- ▶ 词语间的依存关系还可以根据语法关系定义为不同的类型，Carroll 将依存关系细分为 20 种，并给出了关系之间的层级结构。
- ▶ Marneffe 在上述工作的基础上对依存关系进行了进一步的细化，定义了 48 种依存关系，主要分为论元 (Argument) 依存关系和修饰语依存关系两大类。

论元指的是与动词或其他谓词相关的成分，用来表达一个句子中的核心含义。

例如，“小明吃了一个苹果”，小明和苹果就是动词“吃”所需要的两个论元。

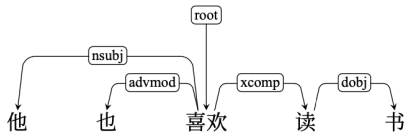
依存语法理论概述

- 依存句法树通常引入root做为句子或者单棵树的主要支配者，是树的根节点。

root 节点：一般情况下，动词是 root 节点的直接从属成分，其余节点应该直接或者间接依存于动词节点。

节点的支配关系：依存句法树的每个节点只能有一个支配者，但可以有多个从属者。

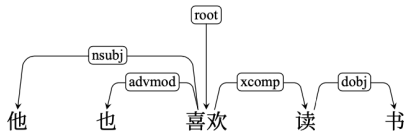
游离节点：不允许存在游离在外的节点。所有节点都必须直接或间接依存于 root 节点。



依存语法理论概述

论元依存关系 (Argument Dependencies)

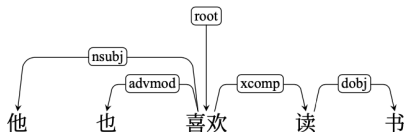
- ▶ **nsubj (主语关系)**，即名词短语是动词的主语。例如，“他”是“喜欢”的主语，nsubj 标记就是指向“他”的依存弧。
- ▶ **dobj (直接宾语关系)**，即名词短语是动词的宾语。例如，“书”是“读”的直接宾语，dobj 标记就是指向“书”的依存弧。
- ▶ **iobj (间接宾语关系)**：名词短语是动词的间接宾语。例子：“他给我书。”中，“我”是“给”的间接宾语。
- ▶ **xcomp (补语关系)**：动词后面需要接一个补语来完成句子意义。例子：“他喜欢读书。”中，“读”是“喜欢”的补语。



依存语法理论概述

修饰语依存关系 (Modifier Dependencies)

- ▶ **advmod (状语关系)**，即副词或者副词性短语对句子中的动词或形容词进行修饰。例如，在句子中，“也”就是“喜欢”的状语，advmod 标记就是指向“也”的依存弧。
- ▶ **amod (形容词修饰关系)**：形容词修饰名词。例子：“他读了一本有趣的书。”中，“有趣”是“书”的形容词修饰语。
- ▶ **nmod (名词修饰关系)**：名词修饰另一个名词。例子：“他读了一本关于语言学的书。”中，“语言学”是“书”的名词修饰语。

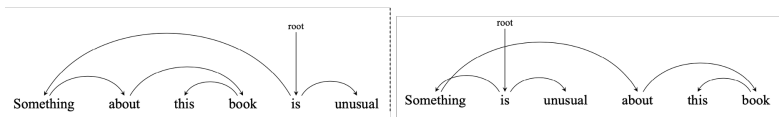


依存语法理论概述

投射性原则是依存语法中的一个重要概念，用于描述依存成分与其中心语或姐妹成分在语序上的关系，可以分为符合投射性原则和违反投射性原则。

- ▶ 在依存层级中如果每个依存成分与其中心语或姐妹成分相邻，那么该依存层级就是符合投射性原则（Projectivity）的，依存树中没有任何两线交叉，因此是投射性的。
- ▶ 如果依存成分的相邻成分使其与中心语分离，那么就是违反投射性原则的。成分“about this book”与其中心语不相邻，因此存在交叉线，违反了投射原则。下图示例一的所有依存成分都与其中心语相邻，没有交叉线。

这种现象在依存句法中通常称为远距离依赖（Long-distance Dependencies），指依存成分与其中心语之间被其他成分分隔开的现象。



3.2 成分句法分析

成分句法分析

成分句法分析 (Constituency Parsing) 是对给定句子根据成分语法中制定的规则构建其所对应的结构树的过程。

成分语法主要包含句法范畴和短语结构规则两个部分。句法范畴又包含短语范畴、词汇范畴和功能范畴。

- ▶ 通常可以用 $X \rightarrow Y Z W \dots$ 表示，与数学系统中的上下文无关文法 (Context-Free Grammar, CFG) 组成非常类似。上下文无关文法是一种形式化系统，用于描述语言的句法结构。它与成分语法中的短语结构规则非常相似。
- ▶ 因此上下文无关文法是目前最常用的模拟英语以及其他语言的成分语法的数学系统。

形式语言

乔姆斯基把语言定义为：按照一定规律构成的句子和符号串的有限或无限的集合。

语言可以被看作一个抽象的数学系统，其中句子和符号串是系统的元素，语法规则是系统的约束条件。

一般地，描述一种语言可以有三种途径：

- ▶ 穷举法：枚举出语言中所有的句子。简单直观，适用于句子数量有限的语言。
- ▶ 文法描述：每个句子用严格定义的规则来构造，利用规则生成语言中合法的句子。能够描述句子数量无限的语言。文法给予句子以结构，各成分之间的结构关系清楚明了。上下文无关文法（CFG）是一种常用的文法描述方法。
- ▶ 自动机法：对输入的句子进行合法性检验，区别哪些是语言中的句子，哪些不是。能够高效地识别语言中的句子。缺点：自动机机械地刻画对输入字符串的识别过程，通常不提供句子的结构信息。有限状态自动机（Finite State Automaton, FSA）是一种常用的自动机模型。

形式文法

形式语法（形式文法）是描述语言结构的形式化系统，是一个四元组 $G = (N, \Sigma, R, S)$

- ▶ N 是非终结符（non-terminal symbol）的有限集合，表示句法范畴或短语结构，用大写字母表示。
例如：NP（名词短语），VP（动词短语），N（名词），介词（P）
- ▶ Σ 是终结符号（terminal symbol）的有限集合，表示语言中的具体词汇，用小写字母表示。
 $N \cap \Sigma = \emptyset$, $V = N \cup \Sigma$ 称为总词汇表，表示所有符号的集合。
- ▶ 重写规则 R （Rules）：描述了如何将非终结符改写为终结符或其他非终结符： $R = \{\alpha \rightarrow \beta\}$, α, β 是由 V 中元素构成的串，且 α 至少应含有一个非终结符号。
- ▶ 初始符 S （Start symbol）： $S \in N$ 称为句子符或初始符，是生成句子的起点。在句子生成过程中，通常从 S 开始，逐步应用重写规则，直到生成终结符串。

形式文法

形式语法

► 1) 形式语法定义

$N = \{S, NP, VP, Det, N, V\}$

$\Sigma = \{ \text{"the"}, \text{"a"}, \text{"boy"}, \text{"apple"}, \text{"eats"} \}$

$R = \{$

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow \text{"the"} \mid \text{"a"}$

$N \rightarrow \text{"boy"} \mid \text{"apple"}$

$V \rightarrow \text{"eats"}$

$\}$

$S = S$

形式文法

形式语法

- 2) 生成句子：从 S 开始，逐步应用重写规则：

S

→ NP VP

→ Det N VP

→ “the” N VP

→ “the” “boy” VP

→ “the” “boy” V NP

→ “the” “boy” “eats” NP

→ “the” “boy” “eats” Det N

→ “the” “boy” “eats” “a” N

→ “the” “boy” “eats” “a” “apple”

上下文无关文法

- ▶ 上下文无关文法是形式语法的一种。

特点：一个规则的左边是一个单独的非终结符，右边是一个或者多个终结符或非终结符组成的有序序列 $(\Sigma \cup N)^*$ 。

α, β, γ 是 $(\Sigma \cup N)^*$ 中的符号串。

上下文无关文法规则： $A \rightarrow \alpha$ ，其中 $A \in N, \alpha \in (\Sigma \cup N)^*$ 。

例如： $S \rightarrow NP VP$,

$NP \rightarrow Det Adj N$,

$Det \rightarrow the$

成分句法分析

- ▶ 对一个句子进行句法分析的过程可以看做对一个句子搜索所有可能的路径空间，从中发现正确的句法树的过程。
- ▶ 句法分析的搜索过程。目标：从所有可能的句法树中，找到符合语法规则且与句子匹配的正确句法树。

约束：

- 1 句子本身：句法树的叶子节点必须与句子中的单词一一对应。
- 2 语法规则：句法树的中间节点与其子节点必须符合语法定义（如短语结构规则）。

成分句法分析

- ▶ 根据搜索方向的不同，句法分析的搜索策略主要分为两种：
 - 1 自底向上 (Bottom-up) 从句子中的单词 (叶子节点) 开始，逐步组合成更大的短语 (中间节点)，直到生成完整的句法树 (根节点)。

优点：只生成与句子单词相关的短语，减少了搜索空间。适用于句子较长或语法较复杂的情况。缺点：可能会生成大量不符合语法规则的中间结果。

例子：句子：“The boy eats an apple.”

自底向上分析过程：

识别单词的词

性：“The”(Det)、“boy”(N)、“eats”(V)、“an”(Det)、“apple”(N)。

组合短语：“The boy” (NP)、“an apple” (NP)。

组合更大的短语：“eats an apple” (VP)。

生成完整的句法树：“The boy eats an apple.” (S)。

成分句法分析

- ▶ 根据搜索方向的不同，句法分析的搜索策略主要分为两种：
 - 2 自顶向下 (Top-down) 从句子的根节点 (S) 开始，逐步扩展成更小的短语 (中间节点)，直到生成与句子单词对应的叶子节点。

优点：生成的中间结果始终符合语法规则。适用于句子较短或语法较简单的情况。缺点：可能会生成大量与句子单词无关的中间结果。

例子：句子：“The boy eats an apple.”

自顶向下分析过程：

从根节点 S 开始，扩展为 NP 和 VP。

扩展 NP 为 Det 和 N。

扩展 VP 为 V 和 NP。

扩展第二个 NP 为 Det 和 N。

匹配叶子节

点：“The”(Det)、“boy”(N)、“eats”(V)、“an”(Det)、“apple”(N)。

成分句法分析

- ▶ 结构歧义是指同一个句子可能有多种合法的句法解析，导致句子的意义不明确。
- ▶ 消除结构歧义是句法分析中最重要的工作之一。
- 1 附着歧义 (Attachment Ambiguity) 是指修饰语（如介词短语、形容词短语等）可以依附于句子中的多个成分，导致句法解析不唯一。

例句：“The boy saw the man with the telescope.”

两种可能的解析：

“with the telescope” 修饰 “the man”（男人带着望远镜）。

“with the telescope” 修饰 “saw”（男孩用望远镜看到了男人）。

- 2 并列连接歧义 (Coordination Ambiguity) 是指并列结构中的成分可以以不同的方式组合，导致句法解析不唯一。

短语：“重要政策和措施”

两种可能的解析：

“重要政策” 和 “措施” 并列。

“政策” 和 “措施” 并列，同时被 “重要” 修饰。

基于上下文无关文法的成分句法分析

在给定上下文无关文法 G 的情况下，对于给定的句子 $W = \{w_1, w_2, \dots, w_n\}$ ，输出其对应的句法结构，通常有两大类搜索方法：自顶向下和自底向上。

- ▶ **自顶向下**搜索试图从根节点 S 出发，搜索语法中的所有规则直到叶子节点，并行构造所有可能的树。
- ▶ **自底向上**的方法是从输入的单词开始，每次都是用语法规则，直到成功构造了以初始符 S 为根的树。

基于上下文无关文法的成分句法分析

CYK 算法

- ▶ CYK 算法 (Cocke-Younger-Kasami) 是由 John Cocke、Daniel Younger 以及 Tadao Kasami 分别独立提出的基于动态规划思想的自底向上语法分析算法。
- ▶ CYK 算法要求所使用的语法必须符合乔姆斯基范式 (Chomsky Normal Form, CNF)，其语法规则被限制为只具有 $A \rightarrow BC$ 或 $A \rightarrow w$ 这种形式， A, B, C 是非终结符。 w 是终结符。
- ▶ 根据 CNF 语法形式，句法树的叶子节点为单词，单词的父节点为词性符号，在词性符号层之上每一个非终结符都有两个子节点。特点：每条规则要么生成两个非终结符，要么生成一个终结符。CNF 简化了句法分析的过程，使得动态规划算法能够高效地应用。

基于上下文无关文法的成分句法分析

CYK 算法

CYK 算法采用了二维矩阵对整个树结构进行编码。对于一个长度为 n 的句子，构造一个 $(n + 1) \times (n + 1)$ 的二维矩阵 T ：

- ▶ 矩阵主对角线以下全部为 0，表示无效区域，主对角线上的元素由输入句子的终结符号（单词）构成。
- ▶ 主对角线以上的元素 T_{ij} 包含由文法 G 的非终结符构成的集合，这个集合表示输入句子中横跨在位置 i 到 j 之间的单词的组成成分。
- ▶ 输入句子中索引从 0 开始，索引位于输入句子的单词之间，也可以看成单词之间的间隔指针。

例如：

0 她 1 喜欢 2 跳 3 芭蕾 4

基于上下文无关文法的成分句法分析

CYK 算法

- ▶ CYK 算法按照平行于主对角线的方向，逐层向上填写矩阵中的各个元素 T_{ij} 。
- ▶ 如果存在一个正整数 $k(i+1 \leq k \leq j-1)$ ，在文法规则集中具有产生式 $A \rightarrow BC$ 并且 $B \in T_{ik}, C \in T_{kj}$ ，那么将 A 合并到矩阵表的 T_{ij} 中。
- ▶ 为了方便根据矩阵 T 输出句法分析结果，在矩阵 T 每个单元 T_{ij} 中不仅记录非终结符的集合，还要记录推导路径。
- ▶ 逐层计算完成后，判断一个句子由文法 G 所产生的充要条件是： $T_{0n} = S$ ，也就是判断句子合法性。

基于上下文无关文法的成分句法分析

CYK 算法

给定如下符合乔姆斯基范式的文法 G 如下：

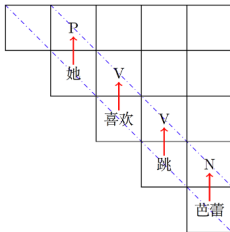
- ▶ 非终结符号集合： $N = \{S, N, P, V, VP\}$
- ▶ 终结符号集合： $\Sigma = \{\text{她}, \text{喜欢}, \text{跳}, \text{芭蕾}\}$
- ▶ 规则集合：

$R = \{ (1) S \rightarrow P VP; (2) VP \rightarrow V V; (3) VP \rightarrow VP N; (4) P \rightarrow \text{她}; (5) V \rightarrow \text{喜欢}; (6) V \rightarrow \text{跳}; (7) N \rightarrow \text{芭蕾} \}$

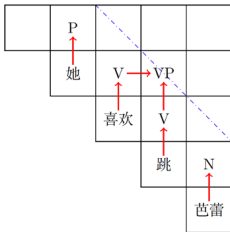
- ▶ 首先初始化矩阵主对角线上的单词以及单词所对应的词性。在此基础上进行第二层 T_{02}, T_{13}, T_{24} 元素的计算，通过规则集合中 $VP \rightarrow V V$ 推导规则，可以得到在 T_{13} 添加 VP 以及相应的路径信息。针对第三层 T_{03}, T_{14} 元素，虽然针对 T_{03} 元素，可以根据推导规则 $S \rightarrow P VP$ 添加非终结符 S ，但是由于非终结符 S 是只能出现在 T_{0n} 中，因此 T_{03} 不添加 S 。 T_{14} 元素中根据通过规则 $VP \rightarrow VP N$ 添加 VP 信息。最后一层 T_{04} 根据 $S \rightarrow P VP$ 规则，添加 S 及其路径信息并完成分析。

基于上下文无关文法的成分句法分析

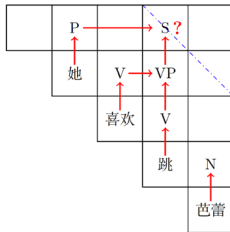
CYK 算法



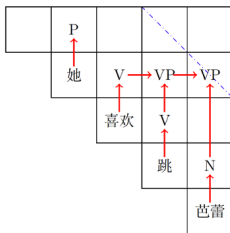
(a)



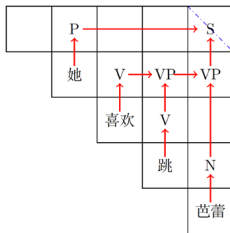
(b)



(c)



(d)



(e)

基于上下文无关文法的成分句法分析

移进-归约成分句法分析算法的基本思想是从左到右扫描输入的包含单词词性对的句子，使用堆栈和一系列的移进（Shift）和归约（Reduce）操作序列构建句法树。

算法初始时堆栈 S 为空，队列 Q 中包含整个句子所有单词。在算法结束时堆栈 S 中包含一个完整的句法树，队列 Q 为空。所采用的操作包含：

- ▶ 移进（Shift）：将非空队列 Q 最左端的单词移入堆栈 S 中。
- ▶ 归约（Reduce）：根据推导规则，根据推导规则右侧所包含非终结符数量，将堆栈 S 中的最顶端相应数量元素移出，然后将利用推导规则产生的新结构压入堆栈中。
- ▶ 接受（Accept）：队列中所有单词都已被移到堆栈中，并且堆栈中只剩下一个由非终结符 S 为根的树，表示分析成功。
- ▶ 拒绝（Reject）：队列中所有单词都已被移到堆栈中，但是堆栈中并非只有一个以非终结符 S 为根的树，并且无法继续归约，表示分析失败。

基于上下文无关文法的成分句法分析

	堆栈	动作	队列
0.	∅	∅	她 喜欢 跳 芭蕾
1.	她	shift	喜欢 跳 芭蕾
2.	P 她	reduce	喜欢 跳 芭蕾
3.	P 喜欢 她	shift	跳 芭蕾
4.	P V 她 喜欢	reduce	跳 芭蕾
5.	P V 跳 她 喜欢	shift	芭蕾
6.	P V V 她 喜欢 跳	reduce	芭蕾
7.	P VP / \ 她 V V 喜欢 跳	reduce	芭蕾
8.	P VP 芭蕾 / \ 她 V V 喜欢 跳	shift	∅

	堆栈	动作	队列
9.	P VP N / \ 她 V V 芭蕾 喜欢 跳	reduce	∅
10.	P VP N / \ 她 VP V 芭蕾 / \ V V 喜欢 跳	reduce	∅
11.	S / \ P VP / \ N 她 VP V 芭蕾 / \ V V 喜欢 跳	reduce	∅
12.	S / \ P VP N / \ 她 VP V 芭蕾 / \ V V 喜欢 跳	accept	∅

图: 移进规约成分句法分析算法分析过程实例

基于上下文无关文法的成分句法分析

如何根据当前堆栈 S 和队列 Q 中的状态，选择下一步的操作是移进-规约分析算法中最重要的部分。由于移进和规约操作并不是完全互斥的，在很多状态下两种操作都可以选择，这就造成了**移进规约冲突**（Shift Reduce Conflict）。

- ▶ 在自然语言这种非确定性语法下，这种冲突不可避免。
- ▶ 在基于规则和策略的移进-规约分析方法中，当有多种规约可能时分析算法需要选择其中某个，当移进和规约都可以时也需要选择执行哪个动作。分析算法可以通过设置执行策略来解决这些冲突：
例如：采用深度优先搜索策略，只有在不能规约时才移进，有多种规约时选择最长的文法规则等策略。
- ▶ 由于移进和规约操作不可撤销，因此即便输入的句子是符合语法的情况下，由于策略选择的问题，也可能分析失败，堆栈中不能规约到只有 S 为根的树的情况。为了解决这个问题也可以采用回溯策略，当分析失败时回溯顺次尝试不同选择。

基于概率上下文无关文法的成分句法分析

- ▶ 基于概率上下文无关文法 (Probabilistic Context-Free Grammar, PCFG) 是上下文无关文法 (CFG) 的扩展, 通过为每条规则赋予概率, 结合了规则方法和统计方法的优点。
- ▶ PCFG 的文法也是由终结符集合 Σ 、非终结符集合 N 、初始符 S 以及规则集合 R 组成。只是在 CFG 的基础上对每条规则增加了概率, 其规则用如下形式表示:

$$A \rightarrow \alpha, p$$

其中 A 为非终结符, $\alpha \in (\Sigma \cup N)^*$ 为终结符和非终结符组成的有序序列集合, p 为 A 推导出 α 的概率, 即 $p = P(A \rightarrow \alpha)$, P 是规则的概率分布, 该概率分布要满足如下条件:

$$\sum_{\alpha} P(A \rightarrow \alpha) = 1$$

对于每个非终结符 A , 其所有规则的概率之和必须为 1。

基于概率上下文无关文法的成分句法分析

- 由于 PCFG 中每个规则中包含了概率信息 $P(A \rightarrow \alpha)$ ，因此可以根据一个句子及其句法分析树计算特定句法分析树的概率、句子的概率以及句子片段的概率。利用特定分析树的概率可以用于消除分析树的歧义。

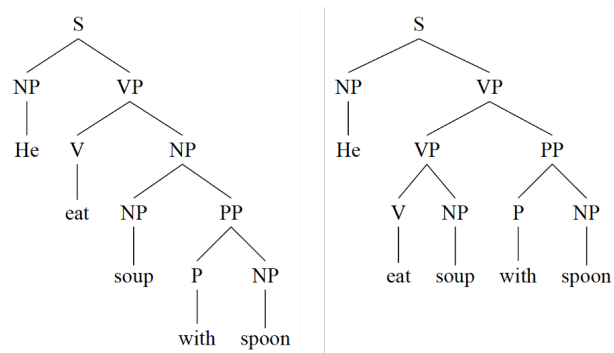


图: 句子 “He eat soup with spoon” 的成分语法树

基于概率上下文无关文法的成分句法分析

如何利用 PCFG 构建句子的最佳树结构？如何得到规则的概率参数？

PCFG 句法分析树概率计算：需满足三个独立假设：

- ▶ 位置不变性 (Place in-variance)：子树的概率不依赖于该子树所在的位置；
- ▶ 上下文无关性 (Context-free)：子树的概率不依赖于子树以外的单词；
- ▶ 祖先无关性 (Ancestor-free)：子树的概率不依赖于子树的祖先节点。

一个特定句法树 T 的概率定义为该句法树 T 中用来得到句子 W 所使用的 m 个规则的概率乘积：

$$P(T) = \prod_{i=1}^m P(A_i \rightarrow \alpha)$$

基于概率上下文无关文法的成分句法分析

- 假设给定如下 PCFG 文法 $G(S)$:

$S \rightarrow NP VP$ 1.0 $VP \rightarrow VP PP$ 0.8 $VP \rightarrow V NP$ 0.2

$NP \rightarrow NP PP$ 0.2 $PP \rightarrow P NP$ 1.0

$NP \rightarrow He$ 0.3 $NP \rightarrow soup$ 0.3 $NP \rightarrow spoon$ 0.2

$V \rightarrow eat$ 1.0 $P \rightarrow with$ 1.0

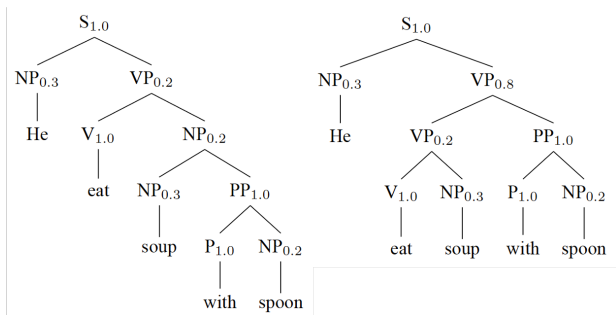


图: 句子 “He eat soup with spoon” 标注推导规则概率的成分语法树

基于概率上下文无关文法的成分句法分析

- ▶ $P(T_{\text{Left}}) = 1.0 \times 0.3 \times 0.2 \times 1.0 \times 0.2 \times 0.3 \times 1.0 \times 1.0 \times 0.2 = 0.00072$
- ▶ $P(T_{\text{Right}}) = 1.0 \times 0.3 \times 0.8 \times 0.2 \times 1.0 \times 1.0 \times 0.3 \times 1.0 \times 0.2 = 0.00288$

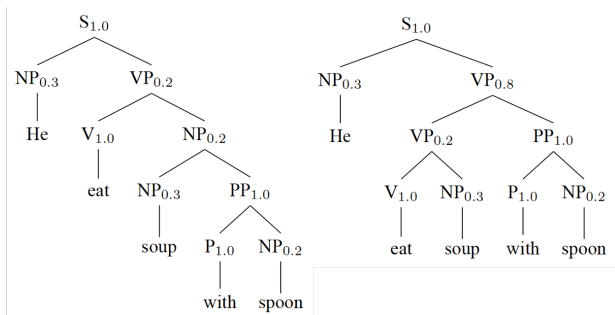


图: 句子 “He eat soup with spoon” 标注推导规则概率的成分语法树

基于概率上下文无关文法的成分句法分析

PCFG 的句子概率计算：在给定 PCFG 文法 G 的情况下，计算给定句子 W 的概率 $P(W|G)$ ，可以采用内向算法 (Inside Algorithm) 或外向算法 (Outside Algorithm)。

- ▶ 采用内向算法，定义内向变量 $\alpha_{ij}(A)$ 为非终结符 A 推导出 W 中子串 $w_i w_{i+1} \dots w_j$ 的概率，即：

$$\alpha_{ij}(A) = P(A \rightarrow w_i w_{i+1} \dots w_j)$$

句子 W 的概率为： $P(W|G) = P(S \rightarrow W|G) = \alpha_{1n}(S)$

通过如下递推公式计算得到：

$\alpha_{ii}(A) = P(A \rightarrow w_i)$ 非终结符 A 直接生成单词 w_i 的概率

$$\alpha_{ij}(A) = \sum_{B,C} \sum_{i \leq k \leq j} P(A \rightarrow BC) \alpha_{ik}(B) \alpha_{(k+1)j}(C) \quad \text{多个单词}(i < j)$$

当 $i \neq j$ 时，子串 $w_i w_{i+1} \dots w_j$ 可以被切分成两端： $w_i \dots w_k$ 和 $w_{k+1} \dots w_j$ ，前半部分 $w_i \dots w_k$ 是由非终结符 B 推导出，后半部分 $w_{k+1} \dots w_j$ 是由非终结符 C 推导出，即

$A \rightarrow BC \rightarrow w_i \dots w_k w_{k+1} \dots w_j$ ，这一推导过程的概率为 $P(A \rightarrow BC) \alpha_{ik}(B) \alpha_{(k+1)j}(C)$ 。

基于概率上下文无关文法的成分句法分析

输入: PCFG $G(S)$ 和句子 $w_1 w_2 \cdots w_n$

输出: $\alpha_{ij}(A), i \leq i \leq j \leq n$

for $i = 1$ to n **do**

$\alpha_{ii}(A) = P(A \rightarrow w_i), 1 \leq i \leq n;$

end

for $j = 1$ to n **do**

for $i = 1$ to $n - j$ **do**

$\alpha_{i(i+j)}(A) = \sum_{B,C} \sum_{i \leq k \leq i+j-1} P(A \rightarrow BC) \alpha_{ik}(B) \alpha_{(k+1)(i+j)}(C);$

end

end

return α

图: 面向 PCFG 句子概率求解的内向算法

基于概率上下文无关文法的成分句法分析

PCFG 的最佳树结构求解：是指对于给定句子 $W = \{w_1, w_2, \dots, w_n\}$ 和 PCFG 文法 G ，求解该句子的最佳树结构，即如何选择句法结构树使得其概率最大：

$$\hat{t} = \underset{t \in \mathcal{T}_G(W)}{\operatorname{argmax}} P(t|W, G)$$

$\mathcal{T}_G(W)$ 表示句子 W 所有符合文法 G 的句法树。

该问题可以通过利用基于概率的 CYK 算法进行。

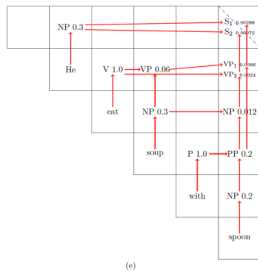
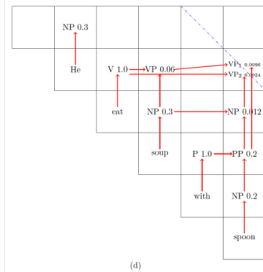
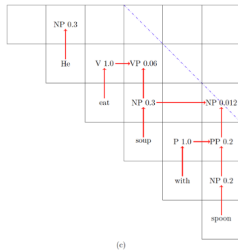
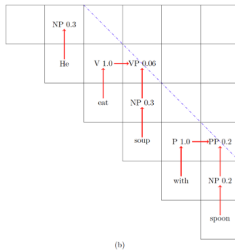
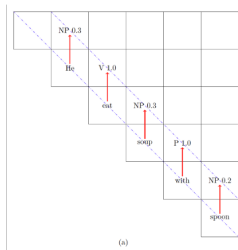
概率 CYK 算法要求所使用的语法必须符合乔姆斯基范式，其规则被限制为 $A \rightarrow BC$ 或 $A \rightarrow w$ 两种形式。与 CYK 算法一样，概率 CYK 算法也将输入的句子表示为单词之间带有索引号的句子形式。

例如：₀ He ₁ eat ₂ soup ₃ with ₄ spoon ₅

基于概率上下文无关文法的成分句法分析

- ▶ 首先初始化矩阵主对角线上的单词以及单词所对应的词性和概率。
- ▶ 之后进行 $T_{02}, T_{13}, T_{24}, T_{35}$ 元素的计算, 通过规则集合中 $VP \rightarrow V NP$ 0.2 的推导规则, 可以得到在 T_{13} 添加 VP 以及相应的路径信息, 其概率为 $0.2 \times 1.0 \times 0.3 = 0.06$ 。
- ▶ 根据 $PP \rightarrow P NP$ 1.0 的推导规则, 在 T_{35} 添加 PP 以及相应的路径信息, 其概率为 $1.0 \times 1.0 \times 0.2 = 0.2$ 。依次逐层进行分析, 在对 T_{15} 进行分析时, 分别根据 $VP \rightarrow VP PP$ 0.8 和 $VP \rightarrow V NP$ 0.2 两个不同的推导规则得到 VP_1 , 其概率为 $0.8 \times 0.06 \times 0.2 = 0.0096$, VP_2 的概率为 $0.2 \times 1.0 \times 0.012 = 0.0024$ 。
- ▶ 据此在 T_{05} 节点可以根据 $S \rightarrow NP VP$ 1.0, 得到两个不同分析结果: S_1 的概率为 $1.0 \times 0.3 \times 0.0096 = 0.00288$ 和 S_2 的概率为 $1.0 \times 0.3 \times 0.0024 = 0.00072$ 。根据所得到的不同分析树的概率大小, 选择最终结果。

基于概率上下文无关文法的成分句法分析



基于概率上下文无关文法的成分句法分析

PCFG 的模型参数学习

- ▶ 在有大规模标注句法树标注的情况下，可以基于最大似然估计，统计非终结符的出现次数进行概率参数估计：

$$P(A \rightarrow \alpha) = \frac{\text{Count}(A \rightarrow \alpha)}{\sum_{\lambda} \text{count}(A \rightarrow \lambda)} = \frac{\text{Count}(A \rightarrow \alpha)}{\text{Count}(A)}$$

$\text{Count}(A \rightarrow \alpha)$ 是指规则 $A \rightarrow \alpha$ 在整个树库中出现的次数， $\text{Count}(A)$ 是指在树库中非终结符 A 出现的次数。

基于概率上下文无关文法的成分句法分析

PCFG 的模型参数学习

- ▶ 在仅有大规模无标记句子的情况下，也可以通过 EM 估计规则的概率参数。

基本思想是首先对给定的 CFG 文法 G 中的每个规则，在满足归一化条件的情况下随机赋值概率值，构造文法 G_0 。在此基础上利用 G_0 对所有句子进行分析，计算出每条规则使用次数的期望值。之后利用期望次数根据最大似然估计原理，得到文法 G 的概率参数更新，记为 G_1 。循环执行该过程直到 G 的概率参数收敛于最大似然估计值。

E-步：利用当前的文法 G_i 估算每条规则出现的期望值。

M-步：重新估算概率得到 G_{i+1} 。

成分句法分析评价方法

PARSEVAL 方法主要包含以下三个指标：

- ▶ (1) 标记精确率 (Labeled Precision, LP)：句法器分析结果中正确的短语结构所占比例，即分析结果与标准句法树中短语匹配的个数占分析器所有输出结果中短语个数的比例：

$$LP = \frac{\text{分析结果中正确的短语个数}}{\text{分析结果中短语总数}} \times 100\%$$

- ▶ (2) 标记召回率 (Labeled Recall, LR)：分析结果中正确的短语结构占标准句法树中短语总数的比例：

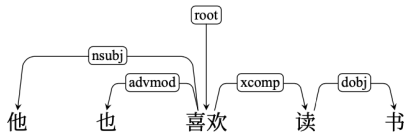
$$LR = \frac{\text{分析结果中正确的短语个数}}{\text{标准结果中短语总数}} \times 100\%$$

- ▶ (3) 交叉括号数 (Crossing brackets, CBs)：成分句法树中所包含的与标准句法树中边界相交叉的短语个数。如果分析器输出的短语边界与标准句法树中的短语边界不完全匹配，且存在交叉，则计为一个交叉括号。CBs 反映了分析器输出结果与标准句法树的结构差异，值越小表示分析器的输出越接近标准结果。

3.3 依存句法分析

依存句法分析

- ▶ 依存句法分析 (Dependency Parsing) 任务目标是依据依存语法理论分析输入句子得到其依存句法结构树。
- ▶ 依存句法理论的基本假设是句法结构由单词和单词之间的依存关系组成。依存关系具有方向性，从中心语成分指向依存成分。依存关系根据中心成分和依存成分之间的关系又可以被定义为不同的依存关系类型。
- ▶ 依存句法结构使用依存图 (Dependency Graph) 进行表示。



依存句法分析

- ▶ 使用 $S = w_0 w_1 \dots w_n$ 表示输入的句子，其中 $w_0 = \text{root}$ 表示虚拟根节点， $w_1 \dots w_n$ 为输入句子中的 n 个单词。
 $R = \{r_0, r_1, \dots, r_m\}$ 表示依存关系类型集合， $r \in R$ 表示在句子中单词之间的依存关系，也叫做边标签 (Arc Label)。
- ▶ 如果依存图 $G = (V, A)$ 对于输入句子 S 和关系集合 R ，是一个从 w_0 出发的有向树，并且包含句子中的所有单词，那么这个依存图 G 就成为形式良好的依存图 (Well-formed Dependency Graph)，也称为依存树 (Dependency Tree)。
- ▶ 对于输入句子 S 和关系集合 R ，所有形式良好的依存图集合记为 G_S 。依存句法分析就是对输入句子 S 根据一定的原则从 G_S 中选择得分最高的依存句法树。

基于图的依存句法分析

- ▶ 基于图的依存句法分析核心是构造评分函数，对句子 S 所有依存句法树 $G = (V, A) \in G_S$ 进行评分。这个评分代表了一个句法树作为句子分析正确结果的可能性。
- ▶ 不同的基于图的分析方法采用不同的假设来计算得分。
- ▶ 基于图的依存句法分析算法通常将对依存句法树的 $G = (V, A)$ 的评分转化为对其树上的边的评分：

$$\text{score}(G) = \sum_{(w_i, r, w_j) \in A} \lambda_{(w_i, r, w_j)}$$

可以将基于图的依存句法分析形式化表示为：

$$\hat{G} = \underset{G=(V,A) \in G_S}{\operatorname{argmax}} \sum_{(w_i, r, w_j) \in A} \lambda_{(w_i, r, w_j)}$$

基于图的依存句法分析

- ▶ 可以证明在依存句法树不考虑投射性 (Projectivity) 的情况下, 对于输入句子 S 的依存句法分析问题等价于基于边评分 $\lambda_{(w_i, r, w_j)}$ 的图 G_S 的**最大生成树** (Maximum Spanning Tree) 寻找问题。
- ▶ 利用**最大生成树算法**得到的依存句法树**不具备投射性**。
- ▶ 针对**具有投射性要求**的依存句法树, 可以利用其与上下文无关语法之间的强相关性, 利用**基于 CYK 算法**等上下文无关语法分析算法进行依存句法树分析。

基于图的依存句法分析

非投射性依存句法分析方法

- ▶ 朱-刘/埃德蒙兹算法 (Chu-Liu/Edmonds) ¹² 方法是一种常见的带权有向图最小/大生成树寻找算法, 因此也常被应用于非投射性依存句法分析。该算法的核心是采用贪心的思想对边进行选择, 利用递归方法来实现整个过程。
- ▶ 输入是待分析句子 $S = w_0 w_1 \dots w_n$ 以及边之间的权重 $\lambda(w_i, w_j) \in \Lambda$
- ▶ w_0 是句子的虚拟根节点, 依存句法树中不存在指向 w_0 的边。

¹Chu and Liu' 65, On the Shortest Arborescence of a Directed Graph, Science Sinica

²Edmonds' 67, Optimum Branchings, JRNBS

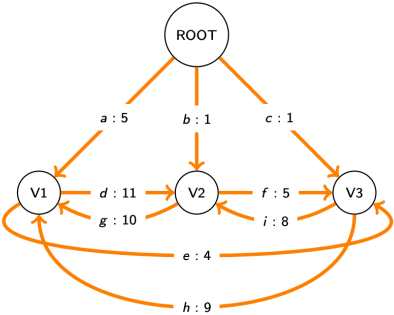
朱-刘/埃德蒙兹算法

- ▶ 每个非根节点都需要恰好 1 条入边。实际上，每个不包含根节点的连通分量都需要恰好 1 条入边。贪婪地选择每个节点的一条入边。
- ▶ 如果这形成了一棵有向树，则满足要求。否则，它将包含一个环 C 。有向树不能有循环，所以不能保留 C 中的每条边， C 中必须排除一条边。
- ▶ C 还需要一条入边，为 C 选择一条入边决定了要排除哪条边。
- ▶ 算法包括两个阶段：
 - 收缩 (Contracting) (递归调用之前的所有操作)
 - 展开 (Expanding) (递归调用之后的所有操作)

朱-刘/埃德蒙兹算法

- ▶ 对于每个非根节点 v ，将 $\text{bestInEdge}[v]$ 设置为其得分最高的入边。
- ▶ 如果形成一个环 C ：
 - ▶ 将循环中的节点收缩成一个新节点 v_C ；
 - ▶ 循环中任何节点发出的边现在的源节点是 v_C ；
 - ▶ 循环中任何节点收到的边现在的目标节点是 v_C ；
 - ▶ 对于循环中的每个节点 u ，以及从循环外部到达 u 的每条边 e ：
将 $e.\text{kicksOut}$ (待排除的边) 设置为 $\text{bestInEdge}[u]$ ，并且
将 $e.\text{score}$ 设置为 $e.\text{score} - e.\text{kicksOut}.\text{score}$ 。
- ▶ 重复，直到每个非根节点都有入边且不形成循环。

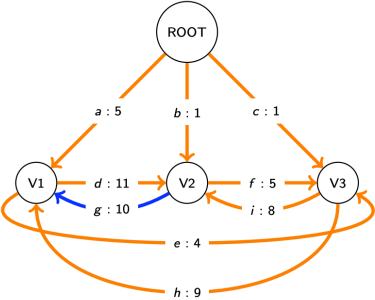
朱-刘/埃德蒙兹算法



	bestInEdge
V1	
V2	
V3	

	kicksOut
a	
b	
c	
d	
e	
f	
g	
h	
i	

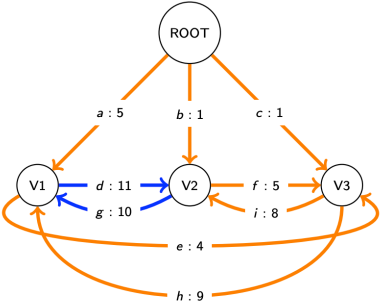
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	
V3	

	kicksOut
a	
b	
c	
d	
e	
f	
g	
h	
i	

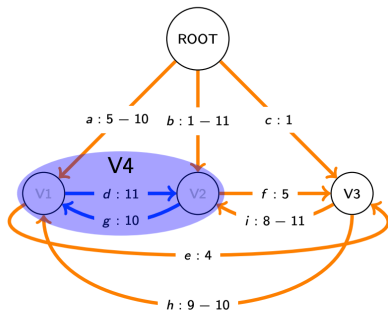
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	

	kicksOut
a	
b	
c	
d	
e	
f	
g	
h	
i	

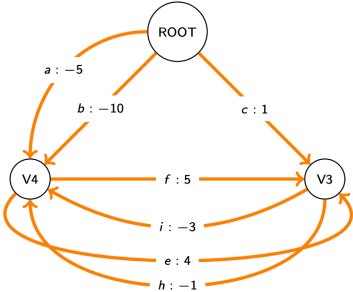
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	

	kicksOut
a	g
b	d
c	
d	
e	
f	
g	g
h	
i	d

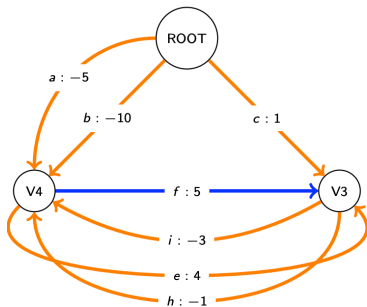
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	
V4	

	kicksOut
a	g
b	d
c	
d	
e	
f	
g	
h	g
i	d

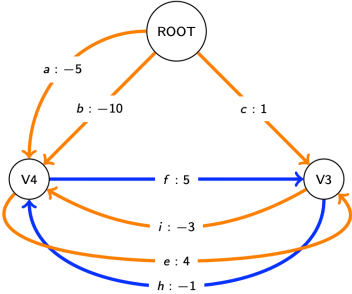
朱-刘/埃德蒙兹算法



	bestInEdge	
V1		g
V2		d
V3		f
V4		

	kicksOut	
a		g
b		d
c		
d		
e		
f		
g		
h		g
i		d

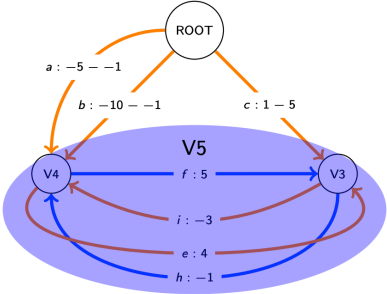
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	f
V4	h

	kicksOut
a	g
b	d
c	
d	
e	
f	
g	
h	g
i	d

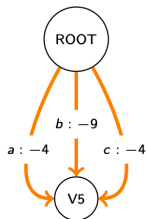
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	f
V4	h
V5	

	kicksOut
a	g, h
b	d, h
c	f
d	
e	
f	
g	
h	g
i	d

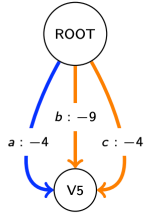
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	f
V4	h
V5	

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

朱-刘/埃德蒙兹算法



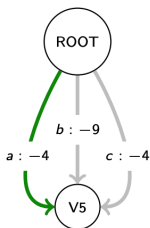
	bestInEdge
V1	g
V2	d
V3	f
V4	h
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

朱-刘/埃德蒙兹算法

- ▶ 在收缩阶段之后，每个收缩节点将恰好有一条 `bestInEdge`。这条边将排除循环内的一条边，打破循环。
 - ▶ 按照添加它们的相反顺序，遍历每个 `bestInEdge` `e`
 - ▶ 锁定 `e`，并从 `bestInEdge` 中移除 `e` 的每条 `kicksOut(e)` 边。

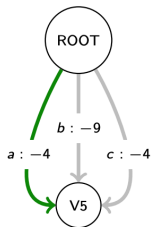
朱-刘/埃德蒙兹算法



	bestInEdge
V1	g
V2	d
V3	f
V4	h
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

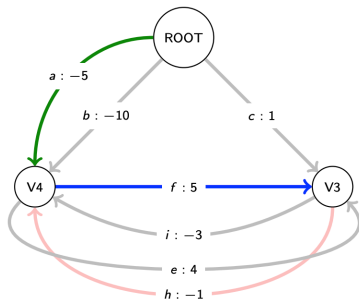
朱-刘/埃德蒙兹算法



	bestInEdge
V1	a g
V2	d
V3	f
V4	a h
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

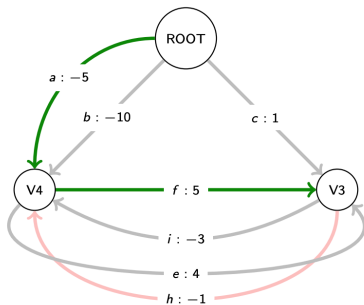
朱-刘/埃德蒙兹算法



	bestInEdge
V1	a g
V2	d
V3	f
V4	a h
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

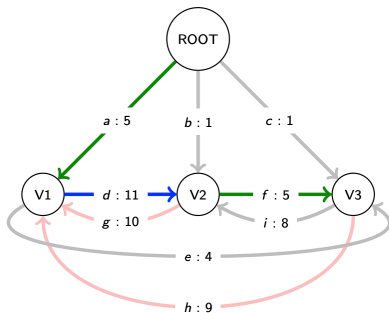
朱-刘/埃德蒙兹算法



	bestInEdge
V1	a
V2	d
V3	f
V4	a
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

朱-刘/埃德蒙兹算法



	bestInEdge
V1	a
V2	d
V3	f
V4	a
V5	a

	kicksOut
a	g, h
b	d, h
c	f
d	
e	f
f	
g	
h	g
i	d

基于图的依存句法分析

- ▶ 根据句子中的单词和权重组成有向图 $G_S = (V_S, A_S)$
- ▶ 针对图 G 中每个顶点选择入边权重最大的边构建子图 $G' = (V_S, A')$ 。如果该子图中没有环，那么该子图就是图 G 的最大生成树。
- ▶ 否则，说明图 G' 中至少包含一个环。那么选择任意一个环 C ，其边集合为 A_C ，将环 C 用一个节点 w_c 来代表，图 G_C 中包含所有在图 G 中但不在环 C 中的节点以及 w_c ， G_C 的边通过以下规则构建：

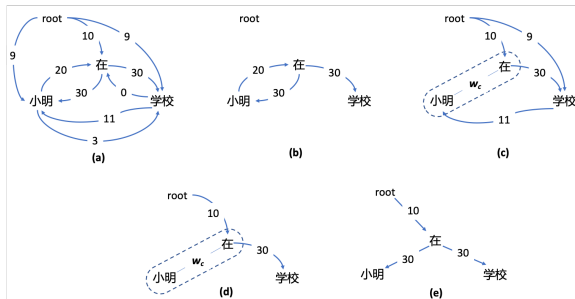
- (1) 对于所有不在环 C 的顶点 w_j ，如果存在一个边 (w_i, w_j) ， w_i 在环 C 中，那么就添加边 (w_c, w_j) 到 G_C 中，定义 $ep(w_c, w_j) = \arg \max_{w_i \in C} \lambda_{(w_i, w_j)}$ ，用于记录环 C 中相对应的顶点，相应的修改 (w_c, w_j) 的边权重 $\lambda_{(w_c, w_j)} = \lambda_{(ep(w_c, w_j), w_j)}$ ；
- (2) 对于所有不在环 C 的顶点 w_i ，如果存在一个边 (w_i, w_j) ， w_j 在环 C 中，那么添加边 (w_i, w_c) 到 G_C 中，定义 $ep(w_i, w_c) = \arg \max_{w_j \in C} [\lambda_{(w_i, w_j)} - \lambda_{(a(w_j), w_j)}]$ ，相应的修改 (w_i, w_c) 的边权重 $\lambda_{(w_i, w_c)} = \lambda_{(w_i, ep(w_i, w_c))} - \lambda_{(a(ep(w_i, w_c)), (ep(w_i, w_c)))} + \sum_{w \in C} \lambda_{(a(w), w)}$ ；
- (3) 对于所有边 (w_i, w_j) ，其顶点 w_i 和 w_j 都不在环 C 中，直接添加该边到图 G' 中，保持原有权重不变。

$a(v)$ 表示节点 v 在环 C 中的父节点

- ▶ 将图 G_C 作为输入，递归调用上述算法得到其最大生成树 $G = (V, A)$ 。之后根据所返回的最大生成树信息对原始图信息进行修正，移除环 C 。

基于图的依存句法分析

► 朱-刘/埃德蒙兹算法生成依存句法树样例



首先根据算法第 1 步，可以获得如图 (a) 所示的有向图，图中顶点为单词，边权重为单词之间构成依存关系的分数。在此基础上，针对每个顶点选择入边权重最大的边构建子图，如图 (b) 所示。在这个子图上存在环，因此将该环所包含的两个顶点合并为虚节点，并根据合并规则构建如图 (c) 所示的子图。在此基础上递归调用第 2 步，每个顶点保留权重最大的入边得到图 (d) 所示的子图。该子图中不包含任何环，因此可以将虚拟节点进行修正，得到对应的最大生成树，如图 (e) 所示。

基于图的依存句法分析

投射性依存句法分析方法

- ▶ 设置虚拟根节点在句子首位的情况下，投射性依存句法分析树等价于嵌套依存句法分析树 (Nested Dependency Trees)。
- ▶ 因此投射性依存句法分析与上下文无关语法具有非常强的关系，很多用于上下文无关语法分析的算法也可以应用于投射性依存句法分析。

基于图的依存句法分析

投射性依存句法分析方法

- ▶ 首先定义动态规划表 $C[s][t][i]$ ($s \leq i \leq t$), 表示投射性句法树以单词 w_i 为根节点覆盖从单词 w_s 到单词 w_t 的句子片段的最高得分。
- ▶ 由此可以得到 $C[0][n][0]$ 表示输入句子 $S = w_0, w_1, \dots, w_n$ 的依存句法树的最高得分。因此对于 $C[s][t][i]$ 可以构造如下递推公式：

$$C[s][t][i] = \max_{s \leq k \leq t, s \leq j \leq t} \begin{cases} C[s][k][i] + C[k+1][t][j] + \lambda(w_i, w_j), & \text{if } j > i \\ C[s][k][j] + C[k+1][t][i] + \lambda(w_i, w_j), & \text{if } j < i \end{cases}$$

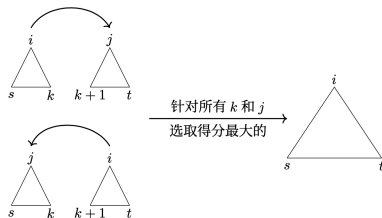


图: 面向依存句法分析的 CYK 算法递推样例

基于图的依存句法分析

投射性依存句法分析方法

- ▶ 由于 $C[s][t][i]$ 中仅保存了子树的最高得分，但是没有保存子树的结构。因此在实际构建过程中，还需要增加树结构矩阵 $A[s][t][i]$ ，用于保存依存句法树结构。通过计算得到最优的 k 和 j 之后， $A[s][t][i]$ 按照如下公式记录树结构：

$$A[s][t][i] = \begin{cases} A[s][k][i] \cup A[k+1][t][j] \cup (w_i, w_j), & \text{if } j > i \\ A[s][k][j] \cup A[k+1][t][i], & \text{if } j < i \end{cases}$$

针对句子 S 得到的依存句法树 $G = (V, A[0][n][0])$ 组成。

基于图的依存句法分析

边评分模型学习方法

- ▶ 边评分模型可以使用基于高维特征向量的线性函数进行建模：

$$\lambda_{(w_i, r, w_j)} = w \cdot f(w_i, r, w_j)$$

- ▶ $f(\cdot)$ 包含了边和输入句子 S 中各类型相关特征：

w_i = 喜欢

w_j = 跳

w_i 的词性 = V

w_j 的词性 = V

r 的依存关系类型 = xcomp

w_{i-1} 的词性 = ADV

w_{j+1} 的词性 = N

w_i 和 w_j 之间距离 = 1

- ▶ 这些特征还可以组合为更复杂的类型例如：
 w_i 的词性 = V & w_j 的词性 = V & w_i = 喜欢
 w_i = 喜欢 & w_j = 跳 & w_i 和 w_j 之间距离 = 1

基于图的依存句法分析

边评分模型学习方法

- ▶ 基于特征向量的线性函数建模的假设下，对于输入句子的依存句法分析转换为了如下问题：

$$\hat{G} = \underset{G=(V,A) \in \mathcal{G}_S}{\operatorname{argmax}} \sum_{(w_i, r, w_j) \in A} \lambda_{(w_i, r, w_j)} = \underset{G=(V,A) \in \mathcal{G}_S}{\operatorname{argmax}} \sum_{(w_i, r, w_j) \in A} \mathbf{w} \cdot \mathbf{f}(w_i, r, w_j)$$

$D = \{(S_d, G_d)\}_{d=1}^{|D|}$ 表示训练语料集合，其中 S_d 表示输入句子，所对应的正确的依存句法树用 G_d 表示。针对输入句子可以利用特征函数 $f(\cdot)$ 构造句子中所有单词对和根据不同依存关系类型输出特征向量。

边评分模型的学习就转换为了权重向量 $\mathbf{w}$ 的学习问题。该问题可以采用感知器算法完成。

基于神经网络的图依存句法分析

基于神经网络的图依存句法分析

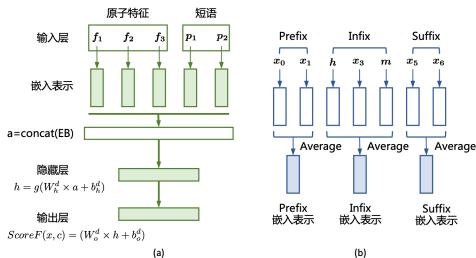
- ▶ 基于图的依存句法分析主要包含边评分模型和句法树生成算法两个部分组成。
- ▶ 其中边评分模型对于分析效果具有决定性的影响。
- ▶ 神经网络方法也可以用于构造边评分。
- ▶ 传统的分析方法依赖人工设计的数百万甚至数千万特征，模型的泛化能力不足，很容易出现过拟合问题。此外，由于特征函数中使用大量的单词关联信息，使得特征空间巨大。神经网络方法可以在一定程度上缓解上述问题，同时也不需要人工设计特征函数。

基于神经网络的图依存句法分析

基于前馈神经网络的方法

模型输入包含：原子特征（Atomic Features）和短语结构（Phrases）。

- 原子特征部分使用单个词和单个词性。通过查找嵌入矩阵 $\mathbf{M} = \mathbb{R}^{d \times |D|}$ 转换为相应的嵌入（Embedding）表示。 $|D|$ 是特征字典大小， d 是特征嵌入表示的维度。



基于神经网络的图依存句法分析

基于前馈神经网络的方法

- 为了更好地建模依存关系的上下文信息，根据所要预测中心词和修饰词在句子中的位置，将句子切分为前缀 (Prefix)、中缀 (Infix) 和后缀 (Suffix)。
- 采用平均池化 (Average Pooling) 的方法构建前缀嵌入表示、中缀嵌入表示和后缀嵌入表示等成短语嵌入 (Phrase Embedding)，并与原子特征一起作为模型输入。

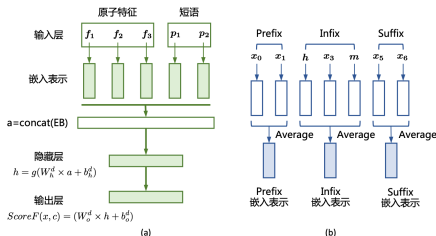
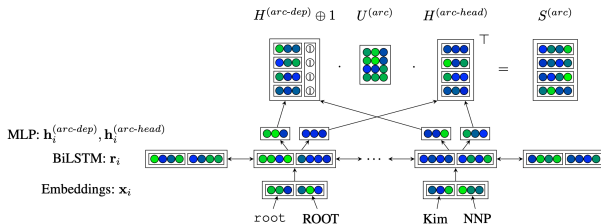


图: h 表示依存关系的中心词 (head), m 表示该依存关系中的修饰词 (modifier)

基于神经网络的图依存句法分析

基于双仿射变换的方法

- ▶ Deep BiAffine Parser 也是采用最大生成树方法构造依存句法树，引入了双向长短时记忆网络，更好地建模句子上下文信息。
- ▶ 考虑到直接使用 BiLSTM 的每个时刻隐藏状态序列作为单词表示可能带来的信息过多，容易造成过拟合并需要更多数量的训练语料的问题，提出了双仿射变换的方法。



基于神经网络的图依存句法分析

基于双仿射变换的方法

- ▶ 使用多层感知器 (MLP) 对 $\mathbf{r}_i$ 用于中心词和修饰词的不同情况，分别进行两种不同的降维：

$$\mathbf{h}_i^{(\text{arc-dep})} = \text{MLP}^{(\text{arc-dep})}(\mathbf{r}_i)$$

$$\mathbf{h}_j^{(\text{arc-head})} = \text{MLP}^{(\text{arc-head})}(\mathbf{r}_j)$$

- ▶ 利用不定类别双仿射分类器对 $\mathbf{H}^{(\text{arc-dep})}$ 额外拼接了一个单位向量，利用矩阵 $\mathbf{U}^{(\text{arc})}$ 进行仿射变换，得到边评分矩阵：

$$\mathbf{S}^{(\text{arc})} = \mathbf{H}^{(\text{arc-dep})} \oplus \mathbf{1} \cdot \mathbf{U}^{(\text{arc})} \cdot \mathbf{H}^{(\text{arc-head})^\top}$$

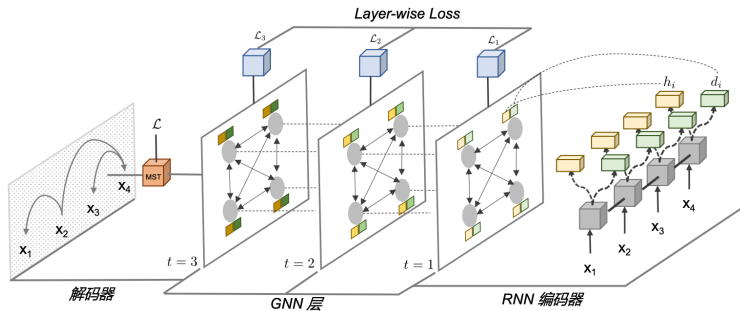
- ▶ 由于依存关系类别数目是确定的，针对类别的分类问题，计算单词 w_i 作为修饰词， w_{y_i} 做为中心词 (head) 的依存关系类别得分：

$$\mathbf{s}_i^{(\text{label})} = \mathbf{r}_{y_i}^\top \mathbf{U}^{(1)} \mathbf{r}_i + (\mathbf{r}_{y_i} \oplus \mathbf{r}_i)^\top \mathbf{U}^{(2)} + \mathbf{b}$$

基于神经网络的图依存句法分析

基于图神经网络的方法

- ▶ 基于图神经网络的依存句法分析算法 (Graph-based Dependency Parsing with Graph Neural Networks, GNNDP)³, 试图将更多的结构化信息引入到节点表示。
- ▶ 基于边评分, 利用贪心或者最大生成树构建依存句法生成树。
- ▶ GNNDP 从表层的单词序列和深层的树结构两个角度进行节点编码。



³Tao Ji, Yuanbin Wu, and Man Lan, ACL 2019.

基于神经网络的图依存句法分析

基于图神经网络的方法

- 单词序列表示使用双向长短时记忆网络 (BiLSTM) 编码单词序列：

$$\begin{aligned}c_i^f &= \text{LSTM} \left(x_i, c_{i-1}^f; \theta^f \right) \\c_i^b &= \text{LSTM} \left(x_i, c_{i+1}^b; \theta^b \right) \\c_i &= c_i^f \oplus c_i^b\end{aligned}$$

由于依存边有向性，因此采用两个多层感知器来生成不同的向量从而区分这两种角色：

$$\begin{aligned}h_i &= \text{MLP}_h(c_i) \\d_i &= \text{MLP}_d(c_i) \\\sigma(i,j) &= \text{softmax}_i(h_i^\top A d_j + b_1^\top h_i + b_2^\top d_j) \triangleq P(i | j)\end{aligned}$$

输出的得分实际上也是依存边 (w_i 支配 w_j) 的概率。

基于神经网络的图依存句法分析

基于图神经网络的方法

- GNNDP 将图神经网络应用到依存句法分析任务，句法分析任务需要在图 G 上同时处理支配词表征 h_i 和从属词表示 d_i ，而不是为每个节点编码一个向量。
为了近似精确的高阶分析，分析器需要每个 GNNs 网络层有关于句子的具体含义。因此，GNNDP 采用完全图（即所有节点都是连接的），并用依存边条件概率设置边权重：

$$\alpha_{ij}^t = \sigma^t(i,j) = P^t(i | j)$$

GNNDP 关注三类高阶信息，即祖父母、孙子和兄弟关系。

基于神经网络的图依存句法分析

基于图神经网络的方法

- 为了纳入祖父母的信息, $\sigma^t(j,i)$ 不仅考虑一跳父子关系 (j,i) , 还考虑两跳 i 的祖父母节点 (用 k 表示)。同样, 为了在 $\sigma^t(j,i)$ 中对两跳的 j 的孙子节点进行编码 (也用 k 表示), 需要聚合邻居节点的从属词表示。

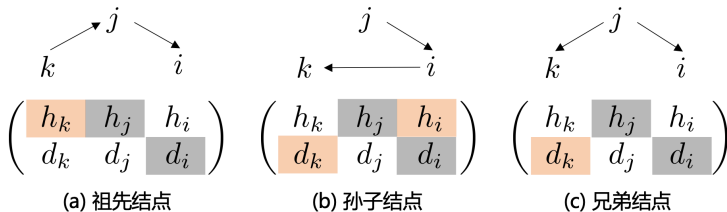


图: 三种类型的高阶信息集成在依存边 (j,i) 中。灰色阴影表示在一阶特征中已经存在的节点表示。橙色阴影表示对于每个高阶特征应该包括哪些节点表示。一阶特征表示节点和边之间的直接关系, 而高阶特征则考虑节点和边的更复杂的组合方式。

基于神经网络的图依存句法分析

基于图神经网络的方法

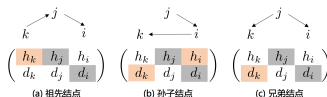
► GNNDP 采用以下协议：

$$\begin{cases} \mathbf{h}_i^t = g\left(W_1 \sum_{j \in N(i)} \alpha_{ji}^t \mathbf{h}_j^{t-1} + B_1 \mathbf{h}_i^{t-1}\right) \\ \mathbf{d}_i^t = g\left(W_2 \sum_{j \in N(i)} \alpha_{ij}^t \mathbf{d}_j^{t-1} + B_2 \mathbf{d}_i^{t-1}\right) \end{cases} \quad (1)$$

为了纳入 i 的兄弟结点（用 k 表示）的信息， $\mathbf{h}_j^t$ 的更新涉及到 $\mathbf{d}_k^{t-1}$ ，这是第二个更新协议：

$$\begin{cases} \mathbf{h}_i^t = g\left(W_1 \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^t \mathbf{d}_j^{t-1} + B_1 \mathbf{h}_i^{t-1}\right) \\ \mathbf{d}_i^t = g\left(W_2 \sum_{j \in \mathcal{N}(i)} \alpha_{ji}^t \mathbf{h}_j^{t-1} + B_2 \mathbf{d}_i^{t-1}\right) \end{cases} \quad (2)$$

$g(\cdot)$ 是一个非线性激活函数。



基于神经网络的图依存句法分析

基于图神经网络的方法，训练目标由两部分组成。

- 第一部分是 GNNs 层后面有一个解码器 (用 τ 表示)，它将对树结构 (使用 $P^\tau(i|j)$) 和边标签 (使用 $P(r|i,j)$) 进行解码。为了预测边标签，使用另一个 MLP 来衡量依存边 (i,j) 具有关系 r 的可能性。其交叉熵损失函数定义如下：

$$\mathcal{L}_0 = -\frac{1}{n} \sum_{(i,j,r) \in T} (\log P^\tau(i | j) + \log P(r | i,j))$$

- 第二个部分是每个 GNNs 层的 $P^\tau(i|j)$ 的准确率进行监督 (只对树结构进行监督，忽略边的标签)。每层交叉熵损失函数定义如下：

$$\mathcal{L}' = \sum_{t=1}^{\tau} \mathcal{L}_t = \sum_{t=1}^{\tau} -\frac{1}{n} \sum_{(i,j,r) \in T} \log P^t(i | j)$$

最终目标是最小化两者的加权组合 $\mathcal{L} = \lambda_1 \mathcal{L}_0 + \lambda_2 \mathcal{L}'$ 。训练完成后，解析器在所有的依存边得分上使用最大生成树算法形成一棵有效的依存树。

基于转移的依存句法分析

- ▶ 基于转移的依存句法分析 (Transition-based Dependency Parsing) 尝试通过模拟人类分析句子的过程来实现依存分析。这种方法依赖于一个模型, 该模型根据当前句子的状态, 通过在两个状态之间进行转移来生成依存树。
- ▶ 转移系统 (Transition System) 包含状态集合 (State 或 Configuration) 以及状态之间的转移动作集合 (Transition)。
- ▶ 标准弧 (Arc-Standard) 转移系统是其中最常用的投射性依存句法分析转移系统之一。

基于转移的依存句法分析

标准弧转移系统定义，状态 $c = (\sigma, \beta, A)$ 有三个部分：

- ▶ σ 表示已处理的单词的堆栈；
- ▶ β 表示尚未处理的单词的缓冲器；
- ▶ A 表示已经构建的依存关系边集合。

对于给定的句子 $S = w_0, w_1, \dots, w_n$ ，其初始状态

$c_0(S) = ([w_0]_\sigma, [w_1, \dots, w_n]_\beta, [\emptyset]_A)$ ， $(\sigma, [], A)$ 则对应终止状态的形式。标准弧转移系统中转移动作包含以下三类：

- ▶ **左弧 (left-arc)**：建立当前栈顶元素和下面一个元素之间的依存关系，并将下面一个元素移入栈中。具体来说，如果栈顶元素是 A ，下面一个元素是 B ，left-arc 的操作会建立 B 到 A 的依存关系，并将 A 弹出栈，将 B 压入栈。
- ▶ **右弧 (right-arc)**：建立当前栈顶元素和栈中下面一个元素之间的依存关系，并将当前栈顶元素弹出栈。具体来说，如果栈顶元素是 A ，下面一个元素是 B ，right-arc 的操作会建立 A 到 B 的依存关系，并将 A 弹出栈。
- ▶ **移进 (shift)**：将缓冲区中的单词移动到栈中。

基于转移的依存句法分析

“她非常喜欢跳芭蕾”的依存句法分析转移动作序列

	σ	β	A
	([ROOT],	[她, ...],	$\emptyset$
SH $\Rightarrow$	([ROOT, 她],	[非常, ...],	$\emptyset$
SH $\Rightarrow$	([ROOT, 她, 非常],	[喜欢, ...],	$\emptyset$
LA _{advmod} $\Rightarrow$	([ROOT, 她],	[喜欢, ...],	$A_1 = (\text{喜欢, advmod, 非常})$
LA _{nsubj} $\Rightarrow$	([ROOT],	[喜欢, ...],	$A_2 = A_1 \cup (\text{喜欢, nsubj, 她})$
SH $\Rightarrow$	([ROOT, 喜欢],	[跳, ...],	A_2
SH $\Rightarrow$	([ROOT, 喜欢, 跳],	[芭蕾舞],	A_2
RA _{dobj} $\Rightarrow$	([ROOT, 喜欢],	[跳],	$A_3 = A_2 \cup (\text{跳, dobj, 芭蕾舞})$
RA _{xcomp} $\Rightarrow$	([ROOT],	[喜欢],	$A_4 = A_3 \cup (\text{喜欢, xcomp, 跳})$
RA _{root} $\Rightarrow$	([],	[ROOT],	$A_5 = A_4 \cup (\text{ROOT, root, 喜欢})$
SH $\Rightarrow$	([ROOT],	[],	A_5

基于转移的依存句法分析

- ▶ 假设存在函数 o ，可以根据当前的状态 c 正确地确定下一步的转移动作 t ，即 $o(c) = t$ 。那么整个句法分析的过程就可以使用非常简单的贪心算法完成。
针对输入句子 S ，首先，构造初始状态 $c_0(S)$ ，调用函数 o 得到下一步转移动作 $t = o(c)$ 。之后，根据转移动作得到下一个状态 $c = t(c)$ 。如此循环直到达到终止状态。
- ▶ 转移动作预测函数可以使用分类器进行建模。
构造一个分类器 $f(c)$ ，输入为状态 c ，输出为其所对应的标准转移动作 $o(c)$ 。
由此，基于转移的依存句法分析问题转化为了典型的机器学习问题。

基于转移的依存句法分析

- 针对状态 c 的表示问题，可以采用从堆栈 σ 、缓存器 β 以及边集合 A 中对特定位置的单词、单词词性、树结构等抽取信息构造特征向量的方法。

	单词	词元	粗粒度词性	细粒度词性	词形特征	依存关系类型
$\beta[0]$	×	×	×	×	×	
$\beta[1]$	×			×		
$\beta[2]$				×		
$\beta[3]$				×		
$ld(\beta[0])$						×
$rd(\beta[0])$						×
$\sigma[0]$	×	×	×	×	×	
$\sigma[1]$				×		
$ld(\sigma[0])$						×
$rd(\sigma[0])$						×

图: Maltparser 采用的构造方法。 $\beta[0]$ 表示缓冲器中最前面的单词， $\sigma[0]$ 表示堆栈最顶端的单词， $ld(x)$ 表示 x 最左面的修饰词， $rd(x)$ 表示 x 最右面的修饰词。利用这些特征模板，可以构造状态的高维向量表示。使用 $f(c)$ 表示特征函数，其输出是状态 c 的特征向量。

基于神经网络的转移依存句法分析

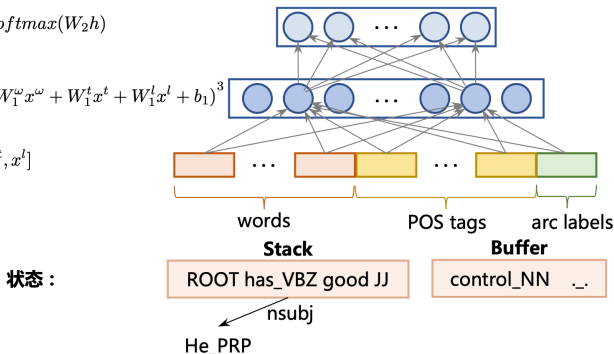
基于转移的依存句法分析通过定义转移系统，将依存句法分析转换为了根据当前状态 $c = (\sigma, \beta, A)$ 预测转移动作 t 的分类问题。

- ▶ 基于前馈神经网络的方法：将当前状态 c 中的堆栈 σ 和缓冲器 β 利用神经网络提取特征，并预测下一步的转移动作的方法。

输出层： $p = \text{softmax}(W_2 h)$

隐藏层： $h = (W_1^\omega x^\omega + W_1^t x^t + W_1^l x^l + b_1)^3$

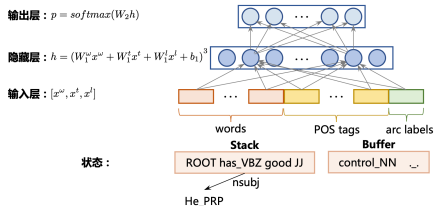
输入层： $[x^\omega, x^t, x^l]$



基于神经网络的转移依存句法分析

词、词性以及边类别嵌入根据设定的上下文连接起来构成 x^w, x^t, x^l 。

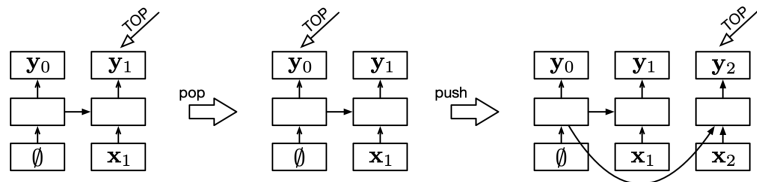
- ▶ x^w 是由堆栈最顶端的 3 个单词和缓冲器中最前面的 3 个单词，以及堆栈中最顶端的 2 个单词的最左、次左、最右和次右的子节点： $lc1(\sigma[i]), lc2(\sigma[i]), rc1(\sigma[i]), rc2(\sigma[i]), i = 1, 2$ ，再加上堆栈中最顶端的 2 个单词的最左的孙子节点和最右的孙子节点： $lc1(lc1(\sigma[i])), rc1(rc1(\sigma[i])), i = 1, 2$ ，共 18 个单词嵌入连接组成。
- ▶ x^t 是由组成 x^w 的 18 个单词的词性嵌入组成。
- ▶ x^l 是 x^w 中除了堆栈和缓冲器的 6 个单词之外的其他 12 个单词对应的边类别嵌入连接组成。



基于神经网络的转移依存句法分析

基于堆栈长短时记忆网络的方法

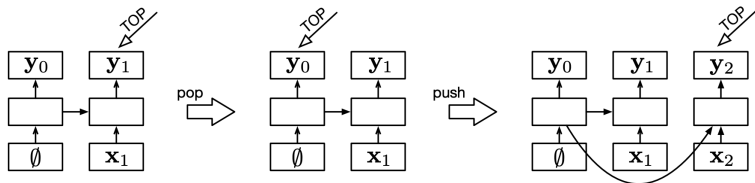
- 基于转移的依存句法分析中需要使用堆栈完成句法分析，因此堆栈长短时记忆网络针对堆栈的压栈（push）和出栈（pop）操作，增加了“栈指针”（stack pointer）来扩充LSTM。



基于神经网络的转移依存句法分析

基于堆栈长短时记忆网络的方法

- ▶ 带有单个元素的堆栈（左侧）、堆栈执行出栈操作的结果（中部），以及此后堆栈使用压栈操作的结果（右侧）。
- ▶ 最底层代表堆栈的内容，是 Stack-LSTM 的输入，中间层是 LSTM 单元，上层是输出层。使用堆栈指针指向的位置作为输出表示。通过上述操作我们可以看到，在执行出栈操作时，Stack-LSTM 中间层 LSTM 单元并不发生改变，仅是修改堆栈指针 TOP 的位置。在执行压栈操作时，新增加的元素增加在最右端，并与前一个 TOP 指针指向的 LSTM 单元连接。



基于神经网络的转移依存句法分析

基于堆栈长短时记忆网络的方法

- 模型中的 S 代表着部分构建的依存子树的堆栈以及其 LSTM 编码， B 代表着尚未处理的单词的缓存以及其 LSTM 编码， A 是表示解析器所采取行动历史的堆栈。这些部分通过线性变换，并通过 ReLU 非线性函数产生解析器状态嵌入 p_t 。该嵌入的仿射变换经过 softmax 层处理后，得到可以采取的解析决策的分布。

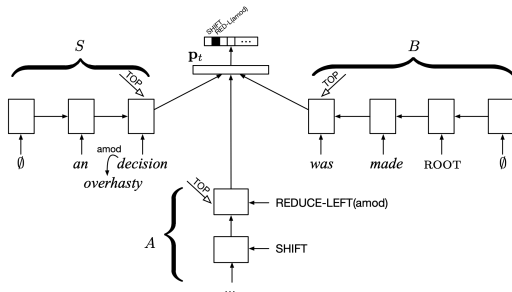


图: 解析句子 “an overhasty decision was made.”

依存句法分析评价方法

- ▶ **依存准确率** (Dependency Accuracy, DA)：正确分析得到其中心词的非根节点词语个数占总非根节点词数的百分比。
 - ▶ 无标记依存准确率 (Un-labeled attachment score, UAS)：表示正确分析得到其修饰词的词语个数占总词数的百分比。
 - ▶ 有标记依存准确率 (Labeled attachment score, LAS)：表示正确分析得到其修饰词以及依存关系的词语个数占总词数的百分比。
- ▶ **根准确率** (Root Accuracy, RA)：正确根节点的个数与句子个数的百分比。
- ▶ **完全匹配率** (Complete Match, CM)：无标记依存结构完全正确的句子占句子总数的百分比。

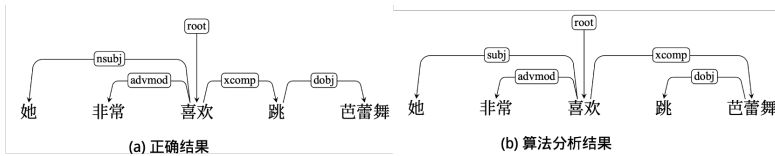


图: DA=2/4, UAS=3/5, LAS=2/5

3.4 句法分析语料库

句法分析语料库

- ▶ 句法分析语料库也称为**句法树库**，包含大规模句子以及其对应句法树的集合。
- ▶ 很多句法分析方法都转换为了有监督机器学习算法，因此句法分析算法的训练和评测都依赖语料库的建设。

语料库名称	单词数量	语法类型	语言
英语宾州树库 (PTB)	117 万	成分语法	英文
通用依存树库 (UD V2.0 CoNLL 2017)	281 万	依存语法	多语言
通用依存树库 (UD V2.2 CoNLL 2018)	1714 万	依存语法	多语言
组合范畴语法树库 (CCGBank)	116 万	组合范畴语法	英文
中文宾州树库 6.0 (CTB 6.0)	78 万	成分语法	中文
中文宾州树库 7.0 (CTB 7.0)	120 万	成分语法	中文
中文宾州树库 8.0 (CTB 8.0)	162 万	成分语法	中文
中文宾州树库 9.0 (CTB 9.0)	208 万	成分语法	中文
中文语义依存树库 (SDP)	52 万	语义依存	中文

句法分析语料库

- 英语宾州树库 (English Penn Treebank, PTB) 是最知名和最常用的短语结构句法树库之一。WSJ(华尔街日报标注) 是英语宾州树库中最常用的部分, 其原始数据来自于 1989 年华尔街日报文章, 按照 PTB(V2) 的标注策略进行标注, 使用括号表示树结构。WSJ 总计包含 49208 个句子, 1173766 个单词, 分成 25 个章 (section), 通常使用 0 到 18 章作为训练集合, 19 到 21 章作为验证集合, 22 到 24 章作为测试集。

```
( (S
  (NP-SBJ (DT Both) (NNS distributions) )
  (VP (VBP are)
    (ADJP-PRD (JJ payable)
      (NP-TMP (NNP Dec.) (CD 4) )
      (PP (TO to)
        (NP
          (NP (JJ limited) (NNS partners) )
          (PP (IN of)
            (NP (NN record) ))
            (NP-TMP (NNP Nov.) (CD 3) )))))
      (NP-TMP (NNP Nov.) (CD 3) )))))
  ( . .) ))
```

句法分析语料库

- 中文宾州树库 (Chinese Penn Treebank, CTB) 是目前最常用的大规模中文短语结构句法标注语料库之一。1998 年开始构建, 2016 年发布了最新的 9.0 版本, 包含中文新闻网站、政府文书、杂志文章、新闻群组、广播对话节目、博客等各类不同来源的 3726 篇文章, 共计 132076 个句子, 2084387 个单词, 3247331 个中文和外文字符。对这些文章中的句子进行了分词、词性标注和成分句法树标注。采用了英语宾州树库的括号结构对树结构进行表示。

```
( ( IP (NP-SBJ (DNP (NP-PN (NR 北海市))
                    (DEG 的))
                    (NP (NN 崛起))))
  (PU , )
  (VP (VC 是)
      (NP-PRD (CP-APP (IP (IP-SBJ (LCP-TMP (NP (NT 近年))
                                              (LC 来))
                                              (NP-PN-SBJ (NR 广西)
                                                         (NN 壮族)
                                                         (NN 自治区))
                                              (VP (PP-DIR (P 对)
                                                         (NP (NN 外))))
                                              (VP (VV 开放))))))
      (VP (VV 取得)
          (NP-OBJ (ADJP (JJ 卓著)
                       (NP (NN 成就)))))
      (DEC 的))
      (ADJP (JJ 重要))
      (NP (NN 标志)
          (NN 之一))))
  (PU 。 )) )
```

句法分析语料库

- 通用依存树库 (Universal Dependencies, UD) 是一个为多种语言开发的跨语言一致的依存句法树库项目。标注方案采用斯坦福通用依存标签、Google 通用词性标签以及形态句法标签集。目前共有超过 300 个贡献者提供了 100 多种语言的 200 多个依存句法树库。树库中包含了原始句子、词语、词性、依存关系以及依存关系类型等信息。

1	同样	同样	ADJ	JJ	_	3	amod	_	SpaceAfter=No
2	的	的	PART	DEC	Case=Gen	1	case	_	SpaceAfter=No
3	手法	手法	NOUN	NN	_	5	nsubj:pass	_	SpaceAfter=No
4	被	被	VERB	BB	Voice=Pass	5	aux:pass	_	SpaceAfter=No
5	用	用	VERB	VV	_	0	root	_	SpaceAfter=No
6	于	于	VERB	VV	_	5	mark	_	SpaceAfter=No
7	现代	现代	NOUN	NN	_	10	nmod	_	SpaceAfter=No
8	的	的	PART	DEC	Case=Gen	7	case	_	SpaceAfter=No
9	摄影	摄影	NOUN	NN	_	10	nmod	_	SpaceAfter=No
10	技术	技术	NOUN	NN	_	5	obj	_	SpaceAfter=No
11	中	中	ADP	IN	_	10	acl	_	SpaceAfter=No
12	。	。	PUNCT	.	_	5	punct	_	SpaceAfter=No

句法分析语料库

- 中文语义依存树库 (Chinese Semantic Dependency Parsing, SDP) 是由哈尔滨工业大学车万翔教授在 SemEval-2016 发布。语料库包含从新闻中选取的 10068 个句子和从小学课文中选取的 14793 个句子。新闻句子平均长度是 31 个词，课文句子平均长度是 14 个词。与传统的依存句法树库不同，中文语义依存树库从语义角度构建依存关系，并依存结构扩展到符合汉语特点的有向无环图-汉语语义依存图结构。定义了 45 个标签用来描述论元 (Argument) 之间的语义关系，19 个标签用来描述谓词 (Predicate) 之间的关系，以及 17 个标签用来提供更丰富的谓词描述。

1	他	他	PN	PN	_	2	Exp	_	_
2	是	是	VC	VC	_	0	Root	_	_
3	研究	研究	NN	NN	_	6	rAgt	_	_
4	机器人	机器人	NN	NN	_	3	Datv	_	_
5	的	的	DEG	DEG	_	3	mAux	_	_
6	专家	专家	NN	NN	_	2	Clas	_	_

本章小结

第三章句法分析

3.1 句法分析概述

3.2 成分句法分析

3.3 依存句法分析

3.4 句法分析语料库